Sequence Closure Plan

Current Algorithm Structure

The current PLACER pipeline is a breakpoint-first long-read TE caller with six main stages:

1. candidate insertion component discovery
2. insertion-fragment extraction
3. fragment-level TE annotation
4. local insertion consensus / assembly
5. TE-internal structural consistency checks
6. final open-set TE state assignment

In code, the main execution path is:

1. `component_module_.build()`
2. `ins_fragment_module_.extract()`
3. `te_classifier_module_.classify()` and `vote_cluster()`
4. `anchor_locked_module_.resolve()`
5. `collect_evidence()`
6. `assemble_component()`
7. `evaluate_post_assembly_te_decision()`
8. `score_placeability()`
9. `genotype_call()`
10. `build_sequence_closure_evidence()`
11. `apply_te_open_set_classification()`

This means PLACER is not only a TE family classifier. It is an insertion-event detector that uses TE sequence evidence as one layer of interpretation.

Stage 1: Candidate Insertion Evidence

PLACER starts from long-read insertion signatures:

- soft clips
- split / supplementary alignments
- long CIGAR insertions

These are clustered into local components and filtered by breakpoint dispersion, split-clip consistency, support size, and low-complexity soft-clip fraction.

Why this is reasonable:

- long-read insertion and SV callers are built on the same evidence sources
- breakpoint spread and support consistency are necessary because repetitive regions produce diffuse soft-clip noise

Stage 2: Insertion-Fragment Extraction

For each component, PLACER extracts insertion-supporting read segments from:

- split-SA evidence
- long insertions
- left/right soft-clipped reads

These fragments are the raw material for both TE classification and local consensus construction.

Why this is reasonable:

- long-read TE callers need explicit sequence segments from reads, not only breakpoint coordinates
- TE family assignment and insertion-hallmark analysis both require sequence context

Stage 3: Fragment-Level TE Annotation

PLACER uses a multi-k-mer TE classifier to score each fragment against a TE reference panel, then aggregates fragment votes into a locus-level TE proposal.

The locus call stores:

- top1 / top2 family
- vote fraction
- posterior margin proxy

- median fragment support
- rescue fraction

Why this is reasonable:

- TE family inference from noisy long-read segments benefits from multiple partial fragments
- multi-fragment voting is more stable than a single best fragment hit

Stage 4: Local Consensus Assembly

PLACER performs POA assembly over insertion fragments and evaluates the local consensus by k-mer identity against its supporting fragments.

Why this is reasonable:

- long-read insertions are noisy at the single-read level
- a local consensus stabilizes sequence interpretation, especially for TE-rich sequence and short inserted bodies

Stage 5: TE-Internal Structural Consistency

PLACER does not stop at family voting. It additionally projects fragments back onto the candidate TE template and checks:

- placement start consistency
- orientation consistency (`theta`)
- anchor length
- split-SA core support
- TE breakpoint core concentration

Why this is reasonable:

- true TE insertions should be internally self-consistent on the TE body
- repetitive false positives often have family-like sequence but poor internal placement coherence

Stage 6: Event-Level Closure and Final State

PLACER now also checks sequence closure at the locus:

- left anchor support
- right anchor support
- dual-anchor support
- empty-span evidence against insertion

The final state is then decided with:

- post-assembly TE decision
- placeability score
- TSD evidence
- sequence-closure evidence
- final open-set TE gating

This is why the current PLACER should be read as:

```
long-read insertion caller + TE sequence interpreter + event-level structural gate
```

Problem

PLACER currently promotes TE calls from a breakpoint-centric component:

1. cluster breakpoint-like reads
2. extract insertion-only fragments
3. assign TE family by fragment voting
4. apply post-hoc gates

This is sufficient to recognize "sequence that looks TE-like", but it does not prove a sequence-level insertion event at a locus. The main failure mode is a clip-dominant component with weak split/indel support that still accumulates a strong TE family vote.

First-Principles Target

A sequence-level self-consistent TE insertion must satisfy:

1. left reference-side anchor support
2. right reference-side anchor support
3. an inserted body sequence between those anchors

TE family annotation is only valid after the event is structurally closed.

Why The Current Structure Is Reasonable

The current algorithm is reasonable because each layer matches a signal class that is widely used or explicitly motivated in long-read TE / insertion literature.

1. Breakpoint-First Candidate Discovery Is Standard

Long-read insertion discovery is normally driven by:

- clipped alignments
- split alignments
- large insertion CIGAR operations
- local read clustering

This is standard in long-read SV callers, including Sniffles2.

Why it supports PLACER:

- TE insertion is a special case of sequence insertion
- the event has to be found before it can be interpreted as a TE

Reference:

- Sniffles2: repeat-aware clustering, consensus, and filtering for long-read SV calling https://www.nature.com/articles/s41587-023-02024-y

2. Sequence Reconstruction Is Necessary For TE Calling

TE callers cannot rely only on breakpoint positions. They must recover some inserted sequence in order to:

- assign family
- determine orientation
- measure truncation
- detect TSD / hallmarks

This is a core idea in TLDR and xTea long-read TE calling.

Why it supports PLACER:

- PLACER extracts insertion fragments and builds a local consensus before final TE interpretation
- this is aligned with the literature direction of "recover insertion sequence, then annotate it"

References:

- TLDR / long-read TE epigenomic profiling and complete insertion sequence recovery https://pubmed.ncbi.nlm.nih.gov/33186547/
- xTea long-read mode / full assembly and annotation of insertional sequences https://pubmed.ncbi.nlm.nih.gov/34158502/

3. TE-Specific Structural Hallmarks Matter

True retrotransposition events are not defined only by family similarity. Relevant hallmarks include:

- target-site duplication
- poly(A) tail
- 5' truncation
- 5' inversion
- transduction

Why it supports PLACER:

- the `anchor_locked` TE-consensus stage is an attempt to enforce internal TE placement coherence
- TSD detection adds a mechanism-level hallmark
- sequence closure adds locus-level self-consistency

References:

- PALMER / long-read mobile element insertion characterization https://pubmed.ncbi.nlm.nih.gov/31853540/
- LINE-1 retrotransposition hallmarks review https://pubmed.ncbi.nlm.nih.gov/26104698/
- poly(A) requirement for efficient LINE-1 retrotransposition https://pubmed.ncbi.nlm.nih.gov/26585388/

4. Open-Set Final Gating Is More Realistic Than Closed-Set Labeling

In repetitive sequence, many candidates will contain partial TE-like sequence without representing a real TE insertion event. A caller therefore needs a final open-set decision:

- `TE_CERTAIN`
- `TE_UNCERTAIN`
- `NON_TE`

Why it supports PLACER:

- this avoids forcing every insertion candidate into a TE family
- the decision can combine sequence evidence, structural evidence, and event closure

5. Current Limitations

The current structure is reasonable, but not complete. The main remaining limitations are:

- assembly is still insertion-fragment-centric, not full haplotype reconstruction
- family posterior is still a proxy from fragment votes, not a full generative likelihood
- TE hallmarks are only partially modeled
- sequence closure is currently a shortest-path read-context validator, not a full filled/empty haplotype model

So the current algorithm should be understood as:

- principled enough to be biologically and methodologically justified
- not yet the final form of a fully sequence-complete TE caller

Shortest-Path Implementation

This implementation keeps the current component discovery and insert-only assembly, but adds a sequence-closure validator that checks whether the support reads contain reference-anchored sequence on both sides of the insertion.

New Evidence Object

Add `SequenceClosureEvidence` with:

- left/right anchor read counts
- dual-anchor read counts
- total anchor-support read counts
- empty-span read counts
- mean anchor identity
- pass / certain / force_non_te flags
- closure QC string

How Closure Is Measured

For each extracted insertion fragment:

- derive local read context immediately left/right of the inserted segment
- match the usable side(s) against the reference around the fragment junction
- count left/right anchored reads after sequence validation

Support interpretation:

- `left + right + enough total` means the locus is structurally closed
- `dual-anchor` or strong split/indel support upgrades closure to `certain`
- too many empty-span reads without anchor support is evidence against a real insertion event

Calling Changes

`TE_CERTAIN` now requires sequence closure certainty.

- no closure: cannot be `TE_CERTAIN`
- weak closure: at most `TE_UNCERTAIN`
- empty-dominant closure failure: can be forced to `NON_TE`

This changes the order of evidence:

1. insertion event closure
2. inserted body TE annotation
3. final TE confidence state

Module-Level Map

Module	Code Path	Input	Output	Why It Exists	Current Weakness	Next Step
Candidate discovery	`component_module_.build()`	raw BAM alignments	breakpoint-centered component	finds insertion-like loci from long-read alignment signatures	clip-heavy repetitive noise can still form components	keep, but rely less on clip-only components downstream
Evidence summarization	`collect_evidence()`	component + fragments + read spans	support counts, breakpoint MAD, consistency metrics	converts raw breakpoint signals into insertion-likelihood features	still component-level, not full event-level	keep as prefilter, do not use as final TE proof
Fragment extraction	`ins_fragment_module_.extract()`	component + supporting reads	insertion-supporting read segments	makes sequence available for TE interpretation and local consensus	fragments are mostly insertion-centric rather than haplotype-centric	gradually move toward filled-event fragments
TE quick annotation	`te_classifier_module_.classify()`	fragments	fragment-level TE hits	gives fast family evidence from noisy fragments	proxy family support can be overconfident in repeats	keep as proposal layer only
Locus TE voting	`vote_cluster()`	fragment hits	top1/top2 TE family and margin	stabilizes family inference across multiple fragments	posterior is a vote proxy, not a full sequence likelihood	keep, but subordinate it to closure and event evidence
TE internal coherence	`anchor_locked_module_.resolve()`	fragments + TE proposal	theta, core set, TE breakpoint structure	checks whether fragments agree on an internal TE placement	only checks TE-body coherence, not full event closure	keep and combine with closure
Local consensus	`assemble_component()`	insertion fragments	local insertion consensus	denoises fragment sequence and recovers inserted sequence proxy	consensus lacks both reference flanks	move toward flank-aware event consensus
Post-assembly TE gate	`evaluate_post_assembly_te_decision()`	TE vote + assembly + component	TE pass / uncertain / fail	combines assembly quality with structural support balance	still allows some clip-dominant loci into uncertain TE path	keep, but closure should dominate certainty
Placeability score	`score_placeability()`	evidence + assembly	tier / support score	prevents weak components from reaching final calling	generic insertion score, not TE-specific	keep as generic insertion filter
Sequence closure	`build_sequence_closure_evidence()`	fragments + read spans + reference	left/right anchor support, empty-span evidence	enforces locus-level self-consistency of the insertion event	current version is read-context based, not a full haplotype closure model	expand toward filled/empty haplotype model

Module	Code Path	Input	Output	Why It Exists	Current Weakness	Next Step
TSD detection	`TSDDetector`	reference window around breakpoint	TSD / DEL / uncertain	adds TE mechanism-level hallmark	reference-window based, not contig-junction aware	later upgrade to contig-junction TSD
Final TE gating	`apply_te_open_set_classification()`	TE proposal + evidence gates	`TE_CERTAIN` / `TE_UNCERTAIN` / `NON_TE`	converts heterogeneous evidence into final open-set status	certainty still uses proxy family evidence more than full event reconstruction	make certainty closure-first, family-second

Failure-Mode Map

Failure Mode	Why It Happens In Current Pipeline	Observable Symptom	Correct Fix Direction
Clip-dominant false positive	component exists, fragments look TE-like, but event is not closed	many `TE_CERTAIN` loci with `TSD_NONE`, weak split/indel support	closure-first gating
Cross-family ambiguity	family vote comes from partial repeat sequence shared by multiple TEs	low posterior margin, top1/top2 family disagreement	keep top2 tracking, lower certainty unless event structure is strong
TE-body coherence without event coherence	fragments align consistently to a TE template but not to a real insertion event	good `theta` / core set but weak locus evidence	combine anchor-locked TE coherence with locus closure, not replace closure
High support but poor mechanistic hallmarks	real insertion-like locus but no TSD / no clear dual-side evidence	many uncertain calls that do not become certain	add more event-level reconstruction before tightening family model
Generic insertion mistaken for TE	insertion assembly exists but family support is weak or spurious	`PASS_INSERTION_TE_UNCERTAIN` over-called in repetitive regions	keep open-set `NON_TE` path and force family after event proof

Priority Order

The shortest-path development order should remain:

1. event closure before family certainty
2. event-aware sequence reconstruction before richer hallmarks
3. richer hallmarks before more sophisticated family posterior

This order matters because:

- if event closure is wrong, family posterior refinement only sharpens false positives
- if reconstruction is weak, hallmarks are measured on the wrong sequence
- hallmarks are only informative once the caller is evaluating the right event

Immediate Roadmap

Step A: Stabilize Current Closure Layer

Goal:

- make `TE_CERTAIN` genuinely dependent on two-sided event support

Done:

- added `SequenceClosureEvidence`
- added final gates driven by closure pass / certainty / force-non-TE

Still weak:

- anchor evidence is computed from read-context validation, not contig closure

Step B: Replace Fragment-Centric Consensus With Flank-Aware Event Consensus

Goal:

- assemble `left flank + inserted body + right flank` rather than only inserted sequence fragments

Expected benefit:

- stronger event self-consistency
- better TSD measurement
- clearer distinction between filled and empty configurations

Step C: Move TSD To Junction-Aware Evaluation

Goal:

- detect TSD from contig-reference junctions instead of only reference windows

Expected benefit:

- reduce false positive TSD-like matches in repetitive sequence
- improve biological interpretability of `TE_CERTAIN`

Step D: Expand TE Hallmark Layer

Goal:

- explicitly measure poly(A), 5' truncation, inversion, and transduction where relevant

Expected benefit:

- improve interpretation of real retrotransposition events
- improve family-specific confidence, especially for LINE-like elements

Step E: Replace Vote-Proxy Family Posterior

Goal:

- move from fragment vote proxies to event-sequence-centered family likelihood

Expected benefit:

- less family collapse in shared-repeat regions
- more stable top1/top2 ranking once event sequence is reconstructed correctly

Code Touch Points

- `include/decision_policy.h`
- `include/pipeline.h`
- `src/pipeline/decision_policy.cpp`
- `src/pipeline/pipeline.cpp`
- `src/component/tsd_detector.cpp`
- `src/main.cpp`
- `scripts/run_sharded_placer.py`
- `tests/test_decision_policy.cpp`

Validation

1. build the project
2. run the unit tests
3. rerun the same BAM through PLACER
4. compare:
    - total TE calls
    - TE_CERTAIN count
    - overlap with TLDR PASS
    - profile of PLACER-unique clip-dominant loci